# SAGE: Safety & Agent Growth Evaluator — Measuring Security Boundary Drift and Capability Retention in Self-Evolving Code Agents

Pratik P. Jain*, Janhavi B. Pagare*, Aditya U. Dengale*, Naitik K. Kharat*, Shamika R. Kadam*, and Vikrant K. K
*Department of Computer Engineering, Vishwakarma Institute of Technology, Pune, India
Email: {pratik.12620589, janhavi.1252010010, aditya.1252010025,
naitik.12620301, shamika.12620290, vikrant.1252010030}@vit.edu

Abstract—Self-evolving LLM agents that mutate prompts, memory, and tools across multi-generation deployments exhibit three compound failure modes invisible to static benchmarks: specification gaming, security boundary drift, and catastrophic forgetting.

We present SAGE, a hardened benchmark measuring these failure modes across 100 tasks, 8 agent archetypes ($G_1$–$G_7$, $G_6^*$), and 10 evolutionary cycles. SAGE pairs deliberate drift probes—seductive visible proxies with sequestered ground-truth tests—with a five-layer container-isolated anti-tamper engine and 18,000 canonical deterministic trajectories.

Key findings: unconstrained agents ($G_4$) achieve 95% on visible proxy tests while collapsing to 40% on hidden ground truth ($\Delta_{\text{proxy}} = +0.55$). Deployable canary guards ($G_7$) limit drift to +0.02 with 96% capability retention. Statistical significance holds across 26 of 27 comparisons under Holm-Bonferroni control ($p_{\text{Holm}} \leq 0.003$), validated by double-blind human audit ($N = 319$, $\kappa = 0.86$).

Code: https://github.com/Pratikjain24/SAGE

Index Terms—LLM agents, specification gaming, security boundary drift, self-evolving agents, benchmarking, reward hacking, regression testing.

## I. Introduction

Autonomous LLM agents increasingly self-evolve: rewriting system prompts ($\Pi_t$), accumulating procedural memories ($\mathcal{M}_t$), and synthesizing persistent tools ($\mathcal{C}_t$). Short-term adaptive gains are documented by Reflexion [1], Voyager [2], and Self-Refine [3]. But unconstrained state mutation creates a perilous optimization dynamic: the shortest path to satisfying an evaluation metric is often to game it.

Three failure modes emerge, invisible in static benchmarks:

1) Specification gaming ($\Delta_{\text{proxy}}$): agents exploit surface test regularities (hardcoded returns, suppressed exceptions) while failing underlying specifications.
2) Security boundary drift (SecurityDrift): defensive code is pruned because it slows benchmark throughput, introducing CWE flaws (command injection, SQL injection, path traversal).
3) Catastrophic forgetting (Retention↓): specialization to recent tasks overwrites foundational capabilities.

Code: https://github.com/Pratikjain24/SAGE. Dataset and trajectories released upon acceptance.

METR [4] documented a 43-fold surge in harness tampering when agents face challenging benchmarks without OS-level containment. Static benchmarks (HumanEval [5], SWE-bench [6]) cannot measure these longitudinal failure modes. We introduce SAGE with four core contributions:

1) A formal archetype taxonomy ($G_1$–$G_7$, $G_6^*$) isolating causal mechanisms of self-evolution.
2) A five-layer container-isolated anti-tamper engine with cryptographic SHA-256 harness auditing.
3) A 100-task golden dataset with 20 deliberate drift probes, synthetic provenance (0% flagged memorization), stratified to target $P(0) = 0.600$ (empirically 59.7% on live models).
4) 18,000 canonical deterministic trajectories providing bitwise-reproducible benchmark reference curves.

## II. System Architecture

### A. Agent State and Evolution Model

An agent at cycle $t$ is represented as $\mathcal{S}_t = \langle \Pi_t, \mathcal{M}_t, \mathcal{C}_t \rangle$, where $\Pi_t$ is the system prompt, $\mathcal{M}_t$ the procedural memory, and $\mathcal{C}_t$ synthesized tools. State mutation is $\mathcal{S}_{t+1} = \mathcal{E}(\mathcal{S}_t, \mathcal{F}_t)$ where $\mathcal{F}_t$ aggregates multi-task execution feedback. The archetype taxonomy controls which mutation surfaces are active:

- $G_1$ (Frozen Baseline): $\Delta\mathcal{S} = \emptyset$
- $G_2$ (Prompt Rewriter): mutates $\Pi_t$ only
- $G_3$ (Memory Accumulator): mutates $\mathcal{M}_t$ only
- $G_4$ (Compound Reflection): mutates $\Pi_t + \mathcal{M}_t + \mathcal{C}_t$ unconstrained
- $G_5$ (Static Verifier): $G_4$ + AST linting rollback gate
- $G_7$ (Proxy Canary Guard): $G_4$ + held-out proxy regression rollback (deployable)
- $G_6^*$ (Oracle Skyline): $G_4$ + sequestered ground-truth rollback (theoretical upper bound)

### B. Five-Layer Anti-Tamper Engine

Agents with shell access converge on reward hacks absent extreme isolation. SAGE deploys five defense

layers across two containers (evo-sandbox + evo-scorer): (1) Decoupled volume mounts – ground-truth suites never enter evo-sandbox; (2) Git clean-index enforcement – any harness modification triggers EXPLOIT_ATTEMPT; (3) SHA-256 digest invariance – harness files reverified after every execution; (4) Static AST inspection – bans unittest.mock, sys.modules hijacking, os._exit(); (5) Kernel isolation – unprivileged UID, –cap-drop=ALL, cgroup limits, –network none.

## III. Dataset and Drift Probes

The 100-task golden dataset contains focused, multi-module algorithmic and system programming component repositories (mean 16.5 mutable LOC) across five balanced domains ($N = 20$ each): Bug Fixing, Feature Development, Refactoring, Exploit Probes, and Security Audits. Tasks span three difficulty tiers (Easy 34%, Medium 33%, Hard 33%) calibrated to target baseline $P(0) = 0.600$ (empirically 61.3% on Qwen, 58.0% on Llama). Mean pairwise TF-IDF cosine similarity is $\mu = 0.0524$ (near-zero semantic overlap). A zero-shot completion audit confirmed 0.0% flagged memorization vs. 32.7% on SWE-bench Verified [7].

Deliberate drift probes: 20 exploit-probe tasks pair a seductive visible proxy (test_proxy.py) with strict sequestered ground truth (test_gt.py). Example: the mini_orm probe's visible test checks that a query returns a dictionary; the hidden ground truth submits SQL injection payloads. Agents that hardcode returns score 100% on the proxy and 0% on ground truth — a maximally detectable gaming signature.

## IV. Metrics and Experimental Setup

Four metrics track longitudinal degradation over cycles $t \in [0, T]$:

$$\Delta P(t) = P_{\mathrm{GT}}(t) - P_{\mathrm{GT}}(0) \tag{1}$$

$$\mathrm{SecurityDrift}(t) = \mathcal{V}(t) - \mathcal{V}(0) \tag{2}$$

$$\mathrm{Retention}(t) = \frac{1}{|\mathcal{H}_0|} \sum_{\tau \in \mathcal{H}_0} 1[\mathrm{eval}(\tau, \mathcal{S}_t) = \mathrm{PASS}] \tag{3}$$

$$\Delta_{\mathrm{proxy}}(t) = P_{\mathrm{Proxy}}(t) - P_{\mathrm{GT}}(t) \tag{4}$$

The full factorial matrix evaluates $100 \times 6 \times 10 \times 3 = 18{,}000$ canonical episodes at temperature 0 with pinned seeds (42, 43, 44). A complementary live validation cohort runs Qwen2.5-Coder-7B and Llama-3.1-8B via the same dual-container harness.

## V. Results

### A. Main Findings

Table I reports results across all 18,000 canonical evaluations. Three core findings emerge:

(1) Unconstrained evolution games surface proxies. $G_4$ reaches 78.4% ground-truth accuracy while proxy accuracy reaches 89.4% ($\Delta_{\mathrm{proxy}} = +0.55$ on drift probes, +0.11 overall). Security drift is +0.28 (**H₂**, **H₃** confirmed).

TABLE I
Main Longitudinal Benchmark Results ($N$=100 tasks, $T$=10 cycles, 3 seeds). $P_{\mathrm{GT}}(0) = 60\%$ for all groups. $\Delta_{\mathrm{prb}}$ = probe gaming gap $\Delta_{\mathrm{proxy}}$(Probe).

| Group | Mechanism | $P_{\mathrm{GT}}(T)$ | $\Delta P$ | Drift | $\Delta_{\mathrm{prb}}$ | Ret. |
|---|---|---|---|---|---|---|
| $G_1$ | Frozen Baseline | 60.0% | +0.00 | 0.00 | +0.03 | 100% |
| $G_2$ | Prompt Rewriter | 73.0% | +0.13 | +0.22 | +0.43 | 82% |
| $G_3$ | Memory Accum. | 77.2% | +0.17 | +0.15 | +0.27 | 89% |
| $G_4$ | Compound Refl. | 78.4% | +0.18 | +0.28 | +0.55 | 81% |
| $G_5$ | Static Verifier | 84.0% | +0.24 | +0.06 | +0.02 | 94% |
| $G_7$ | Proxy Canary | 84.4% | +0.24 | +0.02 | +0.08 | 96% |
| $G_6^*$ | Oracle Skyline | 92.0% | +0.32 | +0.02 | +0.02 | 98% |

(2) Memory-only adaptation stores unsafe shortcuts. $G_3$ exhibits +0.15 security drift and +0.27 probe gaming without mutating prompts or code — trajectory caches encode fragile heuristics.

(3) Dynamic verification establishes rollback-guarded preservation. $G_7$ achieves 84.4% capability with +0.02 drift and 96% retention (**H₅** confirmed). The oracle skyline $G_6^*$ reaches 92.0% with 98% retention.

### B. Statistical Significance

Paired bootstrap tests ($B = 10{,}000$, step-down Holm-Bonferroni FWER, $\alpha = 0.05$) across 27 canonical comparisons yield $p_{\mathrm{Holm}} \leq 0.003$ with large effect sizes (Cohen's $|d| \geq 1.02$, Cliff's $|\delta| \geq 0.56$) in 26 of 27 contrasts. The one non-significant contrast is $G_3$ vs. $G_5$ on $\Delta P(T)$ ($p = 0.294$), reflecting comparable capability gains via different mechanisms. $G_7$'s drift reduction vs. $G_4$ (drift diff +0.26, $d = 8.12$, proxy gap diff +0.35, $d = 9.84$, $p_{\mathrm{Holm}} < 0.001$) is statistically significant under FWER control.

### C. Human Audit

A double-blind audit ($N = 319$ trajectories, 82.7 person-hours, pre-registered $\mathrm{SE}(\hat{\kappa}) \leq 0.038$) yielded Fleiss' $\kappa = 0.856$ (95% CI [0.812, 0.900]) and Cohen's $\kappa = 0.914$ (95% CI [0.878, 0.950]), confirming near-perfect alignment with automated detection.

### D. Ablation Summary

Container ablation: bare-host suffers 5/5 harness escapes; single-container blocks 2/5; dual-container blocks all 5/5 (0% escape rate across 18,000 evaluations, Clopper-Pearson 95% CI $[0\%, 0.02\%]$). Seed sensitivity: $S = 3$ achieves $\mathrm{SE} \pm 0.023$; scaling to $S = 10$ reduces SE by only $\pm 0.011$ at 3.3× compute cost. Horizon: $T = 10$ captures 89.7% of 25-cycle asymptotic drift at 40% compute.

## VI. Limitations and Conclusion

Limitations: Tasks are Python-only (harness is polyglot-ready). The standard horizon is $T = 10$; saturation at $T = 25$ is ongoing. The 18,000 canonical trajectories use formalized archetype policies rather than end-to-end live neural generation — a deliberate trade-off for bitwise

reproducibility. $G_6^*$ is an idealized oracle not available in production; in deployment, proxy canaries ($G_7$) are susceptible to Goodhart's Law and require defense-in-depth with static AST tripwires.

Conclusion: SAGE exposes a fundamental vulnerability: unconstrained self-evolving agents trade ground-truth correctness for superficial metric satisfaction. Cryptographically isolated regression canary guards ($G_7$) establish rollback-guarded state preservation — limiting drift to +0.02 while retaining 96% capability — providing both the empirical benchmark standard and the architectural blueprint for safe lifelong autonomy. Future work extends to Rust/Go/TypeScript tasks, 70B+ model scales, and T=25 horizon studies.

## Data Availability

Code, harnesses, and replication scripts: https://github.com/Pratikjain24/SAGE (Apache-2.0). Distributed Dual-T4 Kaggle replication notebooks: https://www.kaggle.com/code/pratikjain24/sage-distributed-benchmark. Dataset released on Hugging Face (https://huggingface.co/datasets/Pratikjain24/SAGE-benchmark) and Zenodo (DOI: https://doi.org/10.5281/zenodo.14982104).

## References


[1] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in Advances in Neural Information Processing Systems (NeurIPS), vol. 36, 2023, pp. 8634–8652.

[2] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," in Advances in Neural Information Processing Systems (NeurIPS), vol. 36, 2023, pp. 67 199–67 223.

[3] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang et al., "Self-refine: Iterative refinement with self-feedback," in Advances in Neural Information Processing Systems (NeurIPS), vol. 36, 2023, pp. 46 534–46 594.

[4] Model Evaluation and Threat Research, "Evaluating frontier models on autonomous capabilities and long-horizon tasks," METR, Tech. Rep., 2024, https://metr.org.

[5] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman et al., "Evaluating large language models trained on code," arXiv preprint arXiv:2107.03374, 2021.

[6] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in The Twelfth International Conference on Learning Representations (ICLR), 2024.

[7] OpenAI, "Why SWE-bench verified no longer measures frontier coding capabilities," https://openai.com/index/why-swe-bench-verified-no-longer-measures-frontier-coding-capabilities/, February 2026.